Under the hood of WASIp3 Components

Alex Crichton

WASIp3

- Released in June 2026
  - C/C++ support in wasi-sdk-34 (Aug 2026)
  - Rust `wasm32-wasip3` target stable in 1.100.0 (Nov 2026)
- Support `async` for everyone, not just WASI
- Cooperative threading "soon"
- Last (expected) major release before WASI 1.0

Using WASIp2

```
fn main() {
    println!("hi!");
}
```

```
$ cargo build --target wasm32-wasip2
...
$ file target/wasm32-wasip2/debug/test.wasm
target/wasm32-wasip2/debug/wat.wasm: WebAssembly (wasm) binary module version 0x1000d
```

Using WASIp3

```
fn main() {
    println!("hi!");
}
```




```
$ cargo +beta build --target wasm32-wasip3
...
$ file target/wasm32-wasip3/debug/test.wasm
target/wasm32-wasip3/debug/wat.wasm: WebAssembly (wasm) binary module version 0x1000d
```

Using WASIp3

```rust
fn main() {
    std::thread::spawn(|| println!("hi")).join().unwrap();
}
```

```
$ cargo +beta run --target wasm32-wasip3
...
thread 'main' (1) panicked at /.../src/rust/library/std/src/thread/functions.rs:131:29:
failed to spawn thread: Os { code: 58, kind: Unsupported, message: "Not supported" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
Error: failed to run main module `target/wasm32-wasip3/debug/foo.wasm`
```

Using WASIp3

```
fn main() {
    std::thread::spawn(|| println!("hi")).unwrap();
}
```




```
$ export CARGO_TARGET_WASM32_WASIP3_LINKER=$WASI_SDK_PATH/bin/wasm32-wasip3-clang
$ export CARGO_TARGET_WASM32_WASIP3_RUNNER='wasmtime -W component-model-threading'
$ export CARGO_TARGET_WASM32_WASIP3_RUSTFLAGS="\
    -Clink-self-contained=n \
    -Clink-arg=--sysroot=$WASI_SDK_PATH/share/wasi-sysroot/experimental-coop-threads \
    -Clink-arg=-Wl,--export=cabi_realloc"
$ cargo +beta run --target wasm32-wasip3
hi
```

Let's make a component



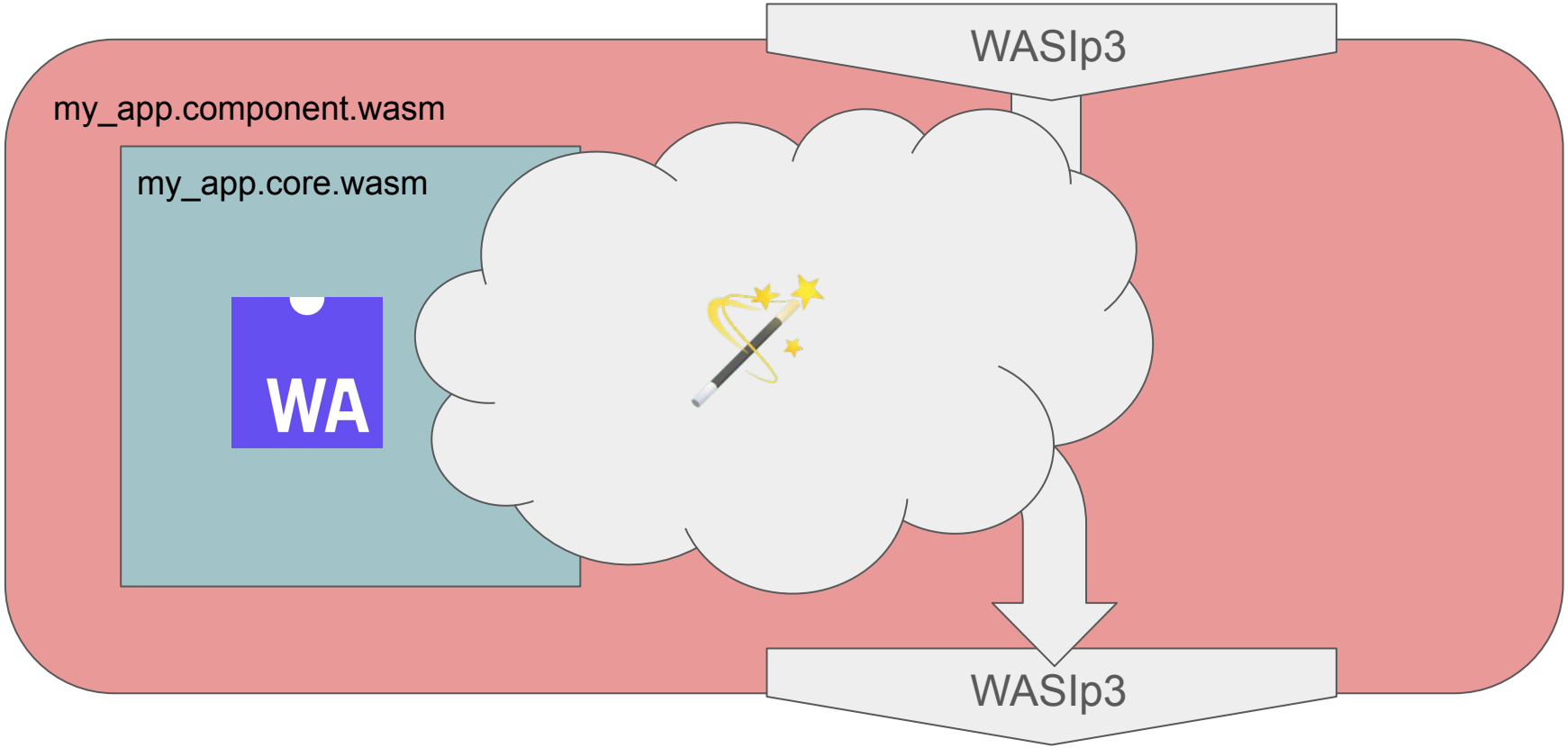
- LLVM
  - Rust
  - C/C++
  - Swift
  - D
- Go
- Kotlin
- Dart
- Scala
- LLVM (again)
  - JS
  - Python
  - Ruby

Let's make a component

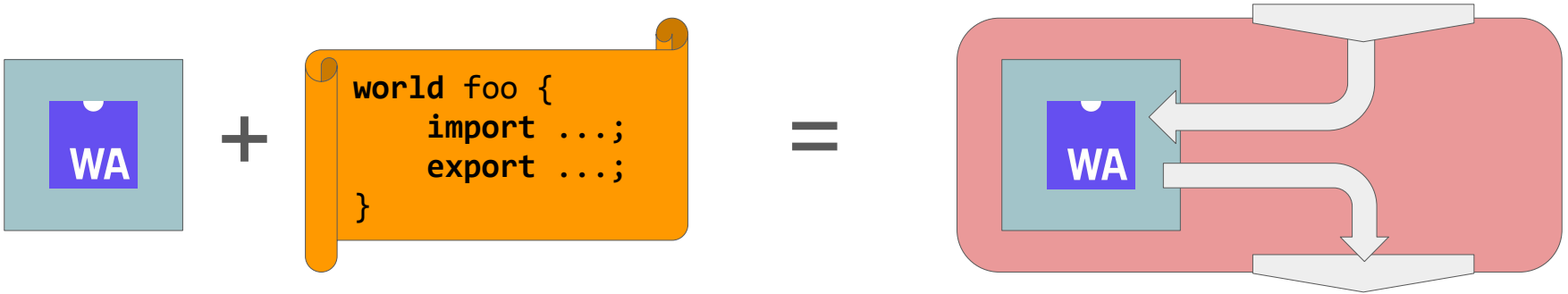


- (none)

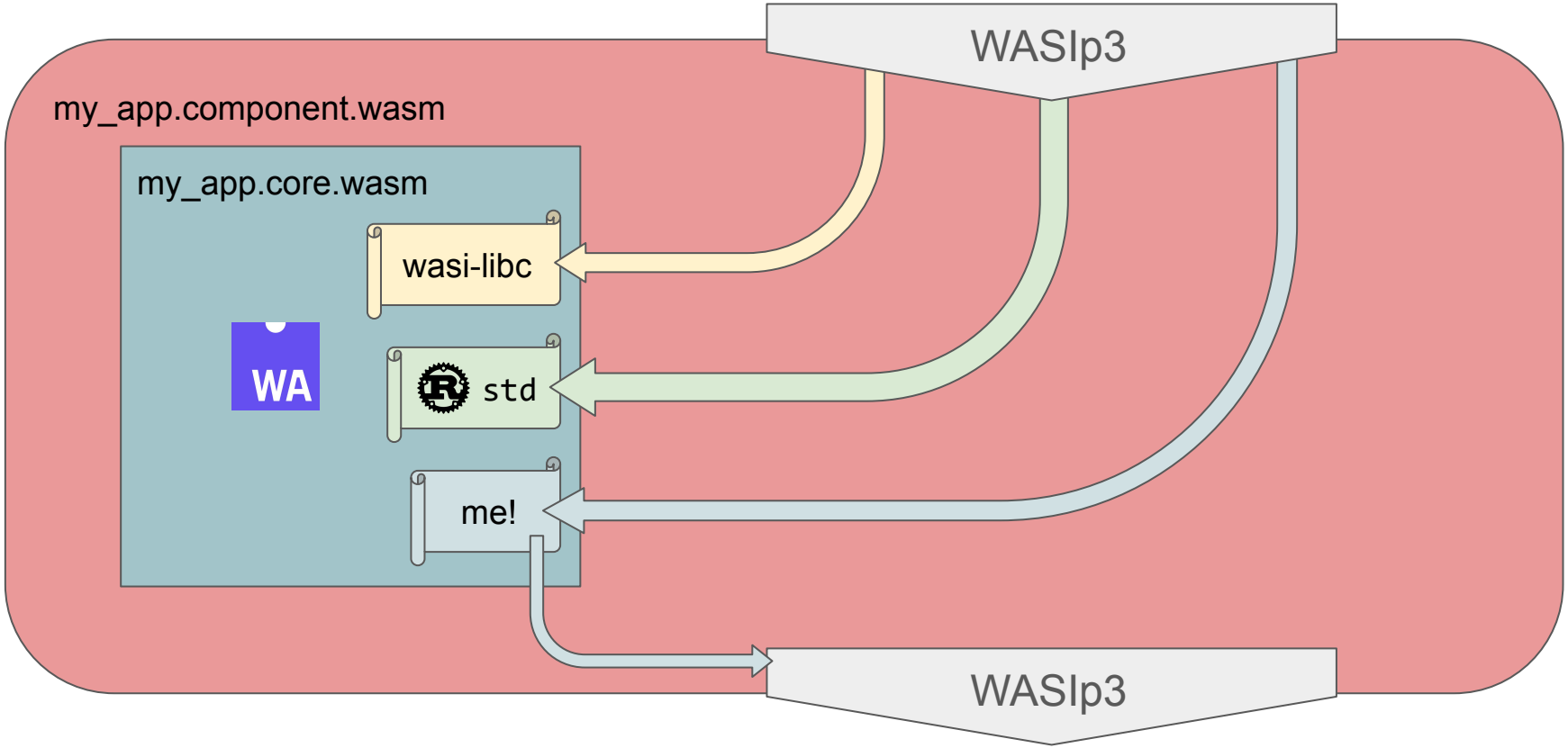
Let's make a component



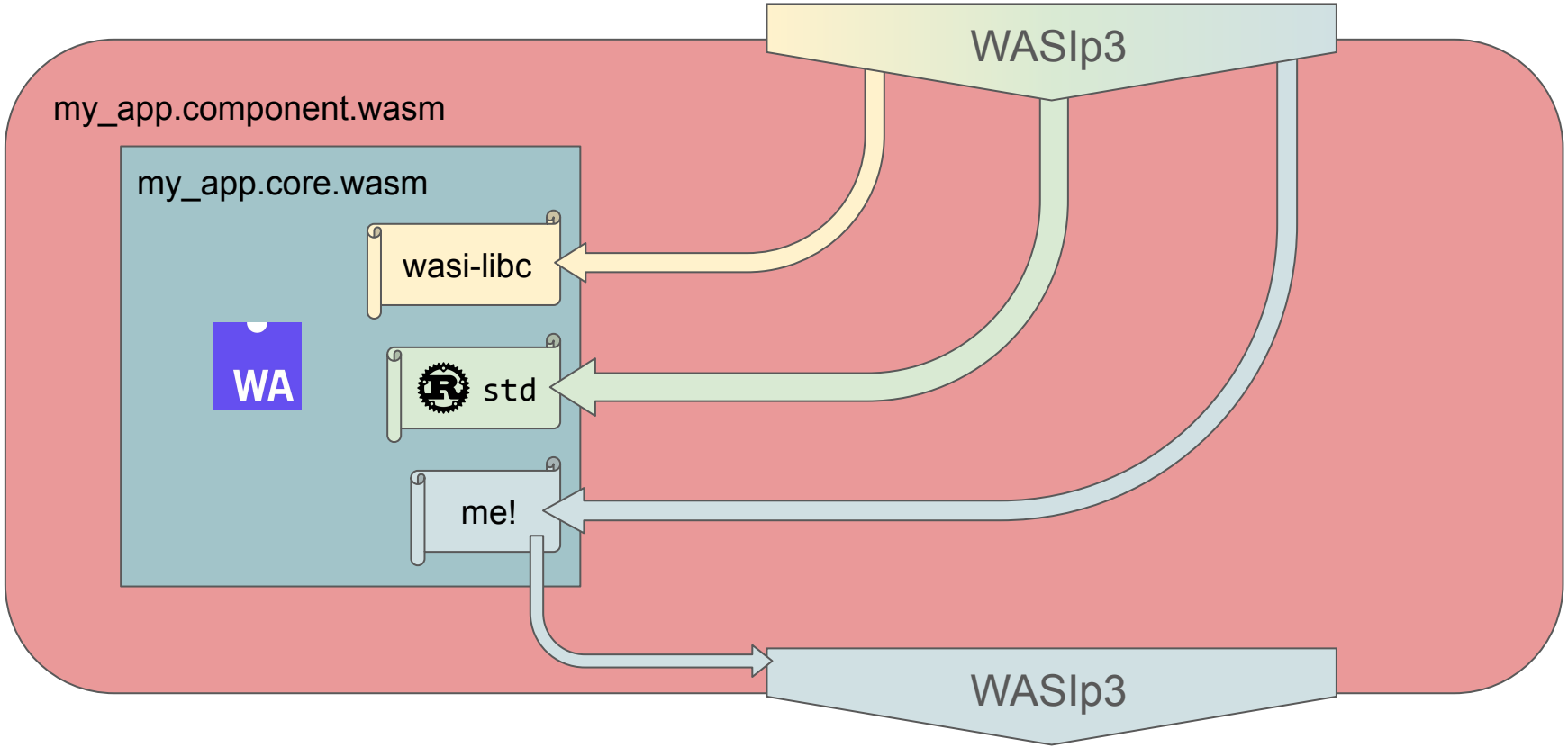
Let's make a component



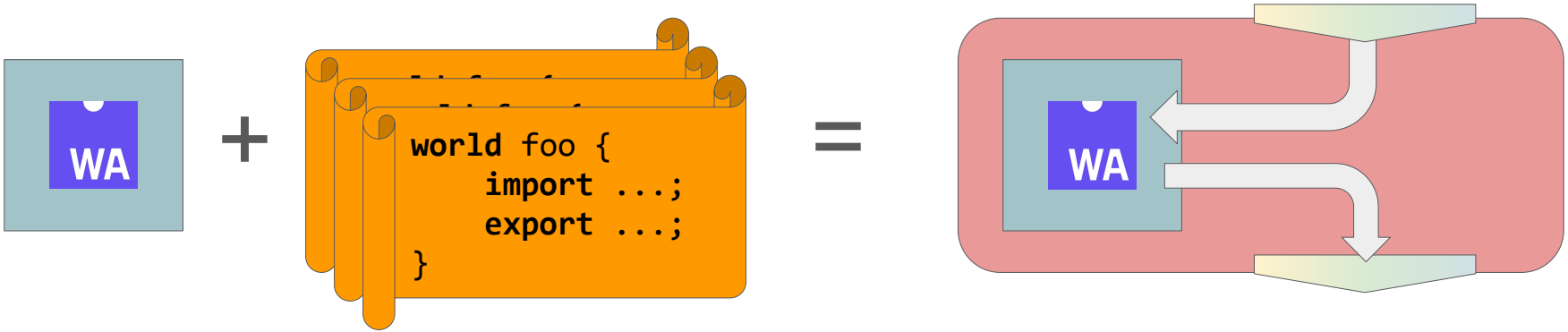
Let's make a component



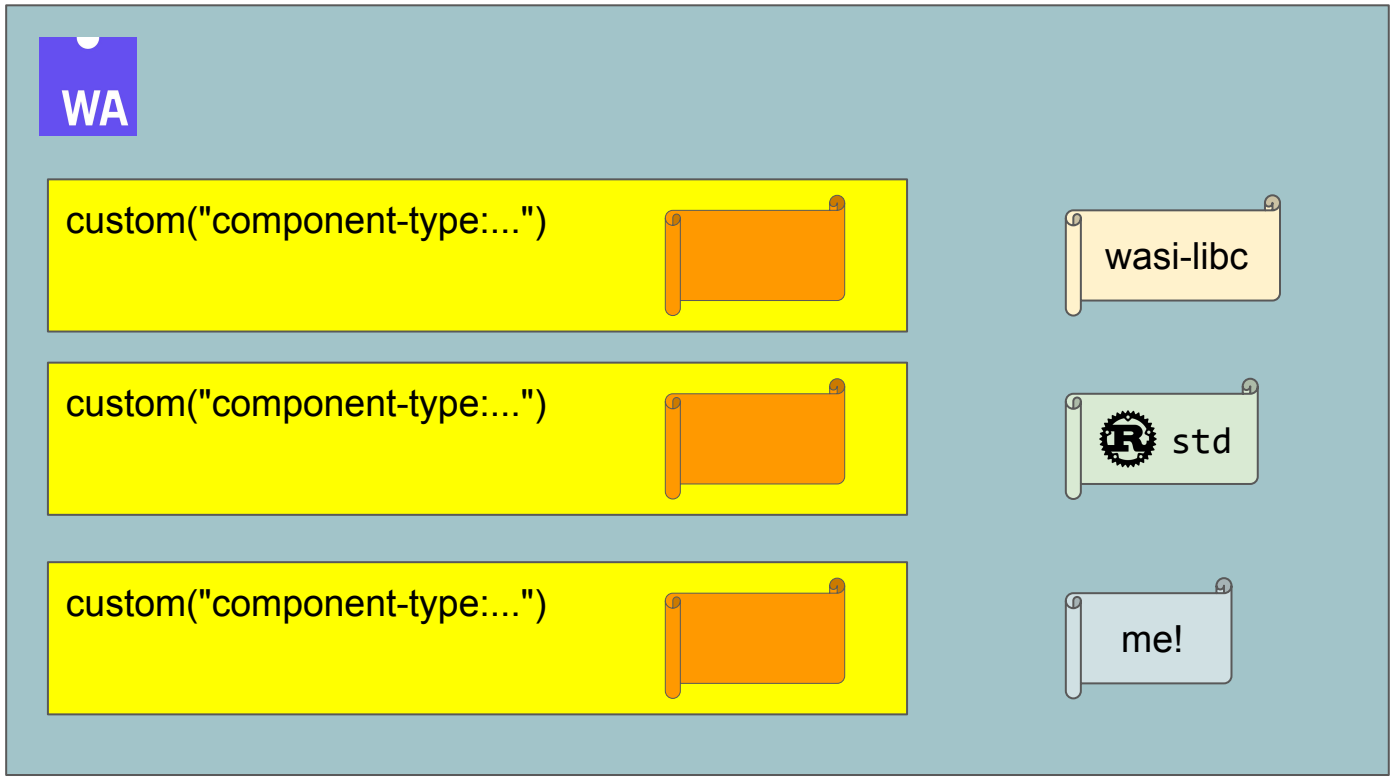
Let's make a component



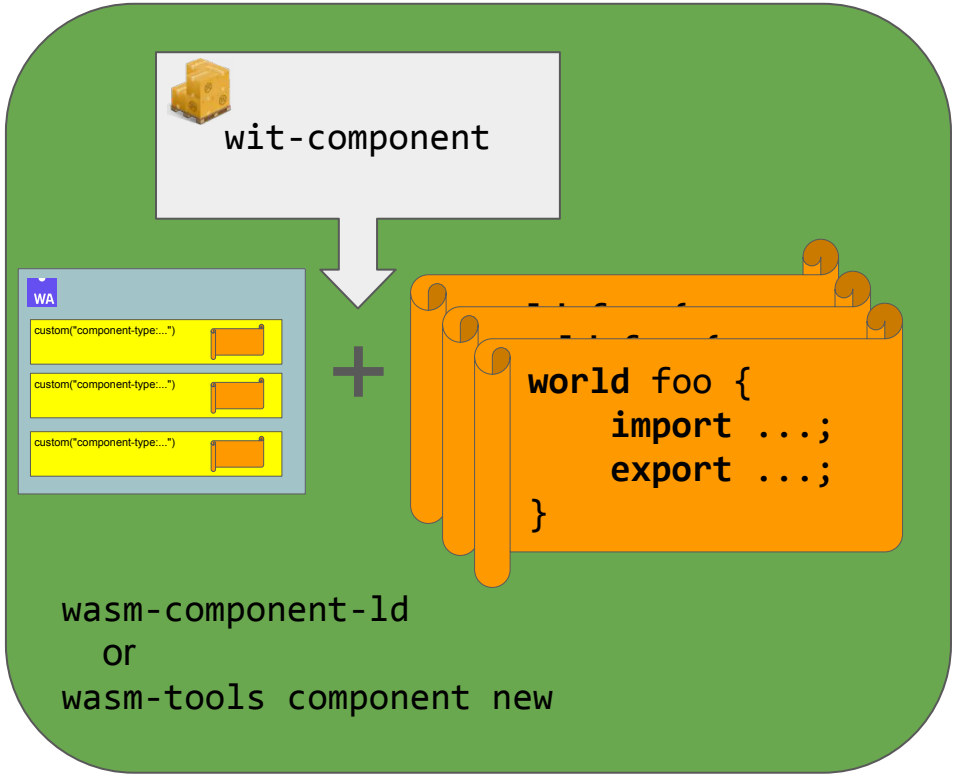
Let's make a component



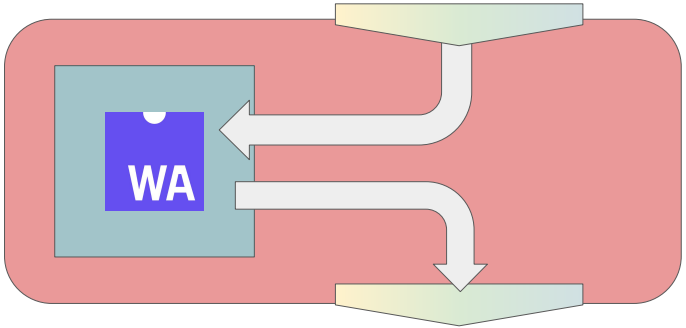
Let's make a component



Let's make a component



=



Naming conventions

```
(module
  (import "$root" "foo" (func))
)
```

```
package my:thing;
world w {
  import foo: func();
}
```

Naming conventions

```
(module
  (import "$root" "foo" (func))
  (import "$root" "[async-lower]bar"
    (func (result i32)))
)
```

```
package my:thing;
world w {
  import foo: func();
  import bar: async func();
}
```

Naming conventions

```
(module
  (import "$root" "foo" (func))
  (import "$root" "[async-lower]bar"
    (func (result i32)))
  (import "$root" "bar" (func))
)
```

```
package my:thing;
world w {
  import foo: func();
  import bar: async func();
}
```

Naming conventions

```
(module
  (import "$root" "foo" (func))
  (import "$root" "[async-lower]bar"
    (func (result i32)))
  (import "$root" "bar" (func))
  (import "$root" "[thread-yield]"
    (func (result i32)))
)
```

```
package my:thing;
world w {
  import foo: func();
  import bar: async func();
}
```

Naming conventions

```
package my:thing;
world w {
  import foo: func(x: stream<u8>);
}
```

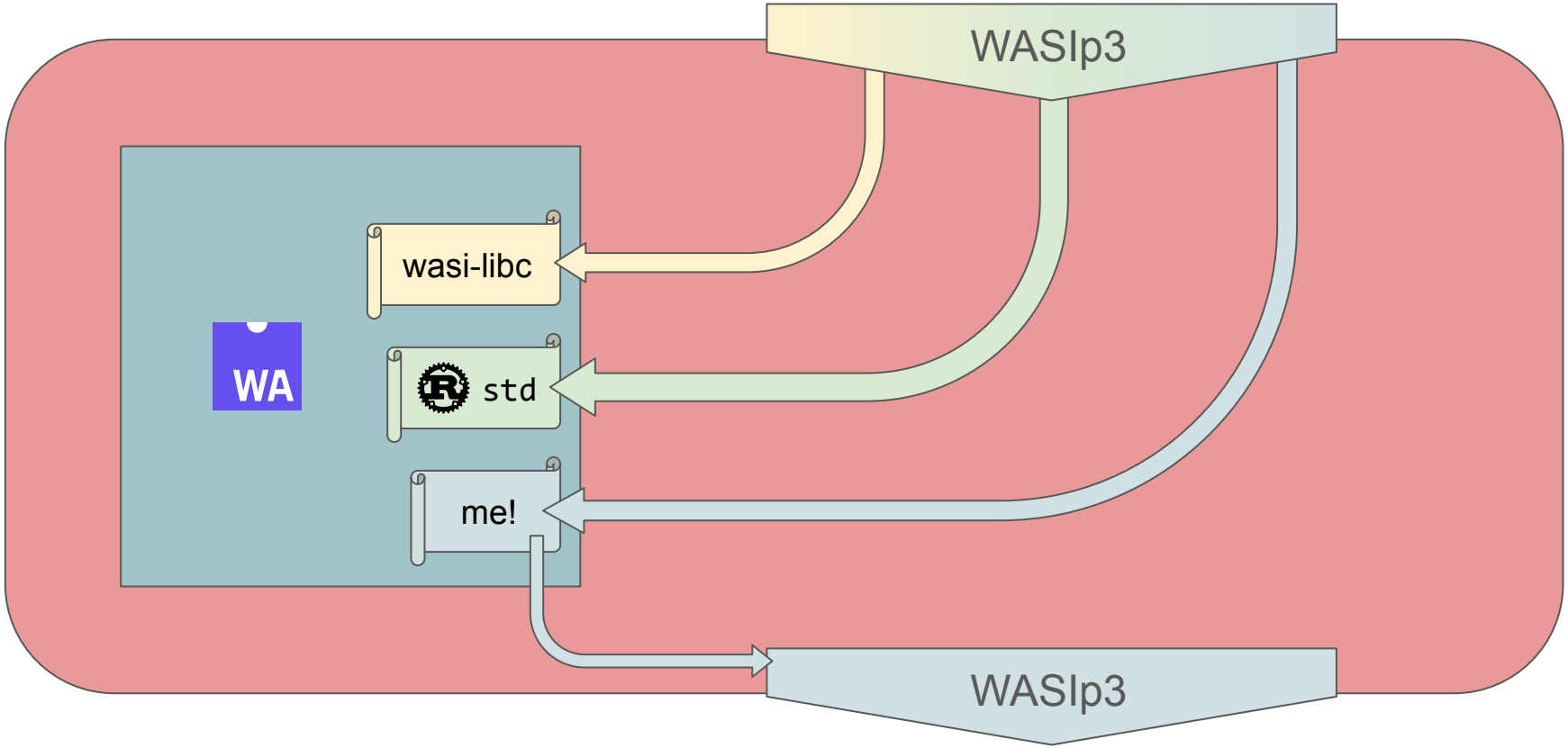
```
(module
  (import "$root" "[stream-new-0]foo" (func (result i64)))
  (import "$root" "[stream-drop-readable-0]foo" (func (param i32)))
  (import "$root" "[async-lower][stream-read-0]foo" (func (param i32 i32 i32) (result i32)))
)
```

Naming conventions

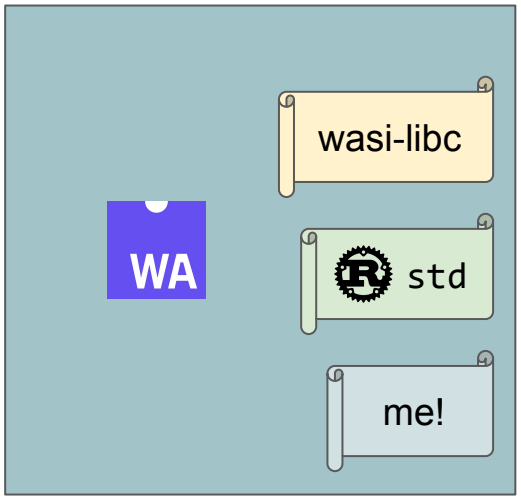
```
(module
  (func (export "[async-lift]foo")
    (result i32)
    ...
  )
  (func
    (export "[callback][async-lift]foo")
    (param i32 i32 i32)
    (result i32)
    ...
  )
)
```

```
package my:thing;
world w {
  export foo: async func();
}
```

Popping the hood



Popping the hood




```
wit-bindgen c wasip3.wit
```

```
[dependencies]
wasip3 = "0.9.0"
```

```
wit-bindgen rust wasip3.wit
```

```
wit_bindgen::generate!("my-async.wit");
```

Structure of a wasip3 component

- Bindings generator used for application code for custom interfaces
- Standard libraries (libc, language, etc.) have bindings to wasip3
- Compilers link everything together into a core wasm
  - Includes custom sections for all WITs used to generate the component
- WITs + core wasm assembled into a component
- Lifts, lowers, and intrinsics coordinated through naming conventions and canonical ABI

Let's add threads! 🧵

- Cooperative threading brings in new component model intrinsics
  - `thread.new-indirect`
  - `thread.{index,resume-later,suspend,yield}`
  - `thread.{suspend,yield}-then-{resume,promote}`
- ...
- profit?



Problem 1: stacks

- Core wasm uses an in-wasm linear memory stack pointer
  - Historically stored in a `global` in the module
- Thread A can block, B resumes and pushes some frames, then A resumes and returns.
- Conclusion: cannot use a `global` for the stack pointer
- Solution: use `context.{get,set} 0` intrinsics

Problem 1: stacks

- Sy Brand implemented support in LLVM for an alternative stack (and TLS) ABI in LLVM/`wasm-ld`
  - `#define __wasm_libcall_thread_context__`
  - LLVM 23+, scoped to the `wasm32-wasip3` target

```
(module
  (import "env" "__wasm_get_stack_pointer" (func (result i32)))
  (import "env" "__wasm_set_stack_pointer" (func (param i32)))
)
```

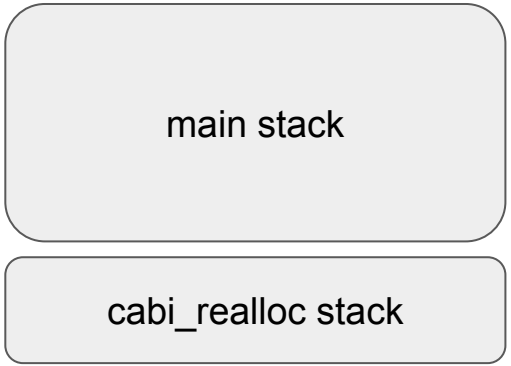
- Hooked up in wit-component to context intrinsics
- Initialized... somewhere?

Problem 1: stacks (part 2)

- Async task slot 0 always starts as 0
  - Even for cabi_realloc, resource destructors, etc
- Can't just use the initial stack pointer, something may already be on the stack
  - Stack usage is always LIFO still, but unlike with a `global` there's no way to know the last stack pointer
- Solution: `__wasm_task_hook`
  - New `wit-component` convention
  - Lifted functions have the user code sandwiched by hook invocations
  - Libc dynamically keeps track of who owns the main stack, allocating new ones as needed

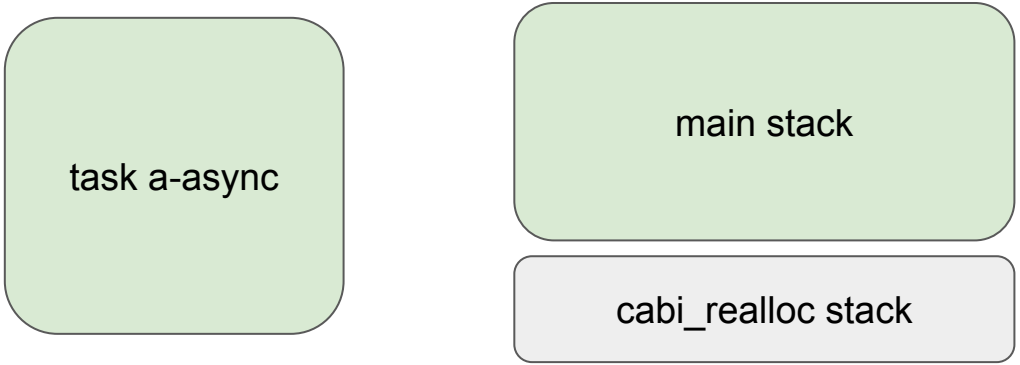
Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



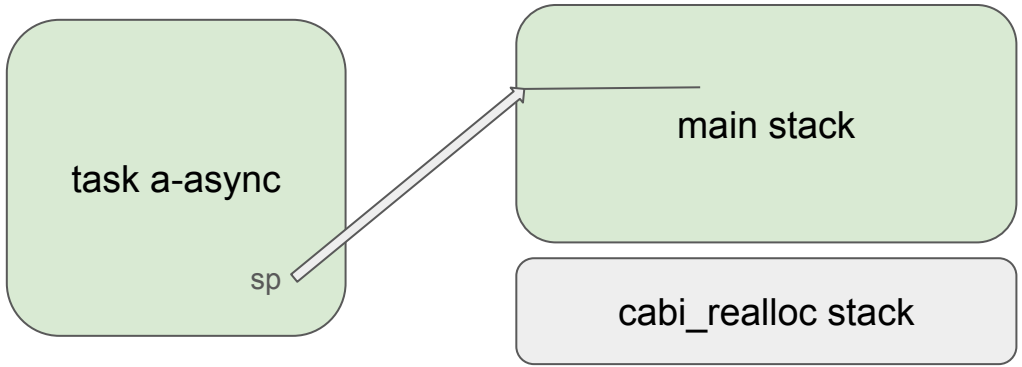
Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



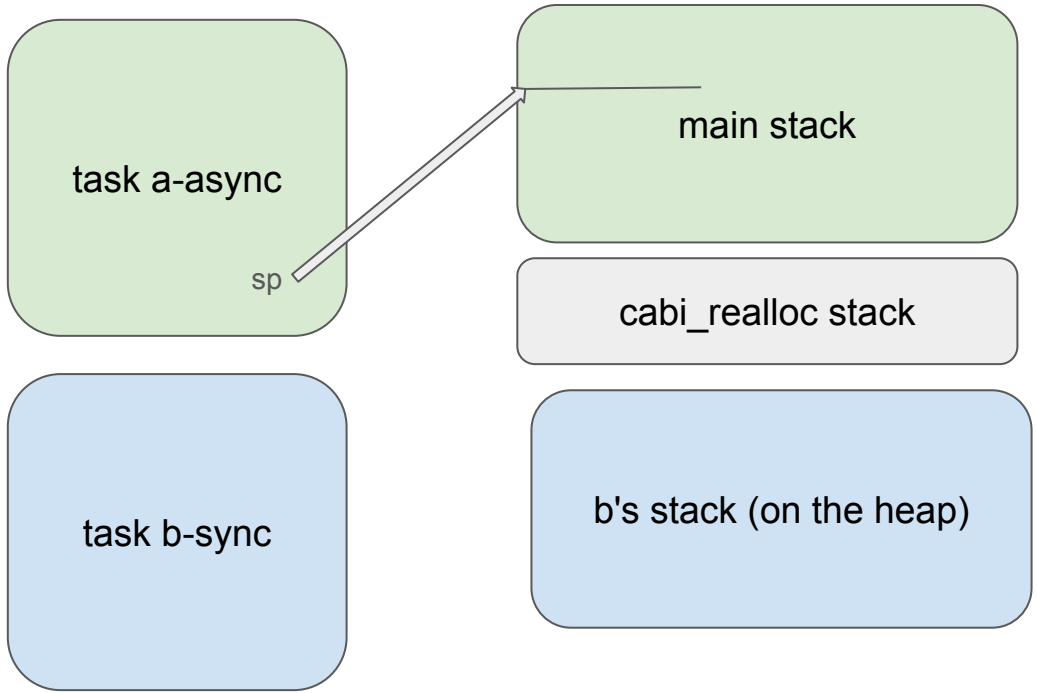
Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



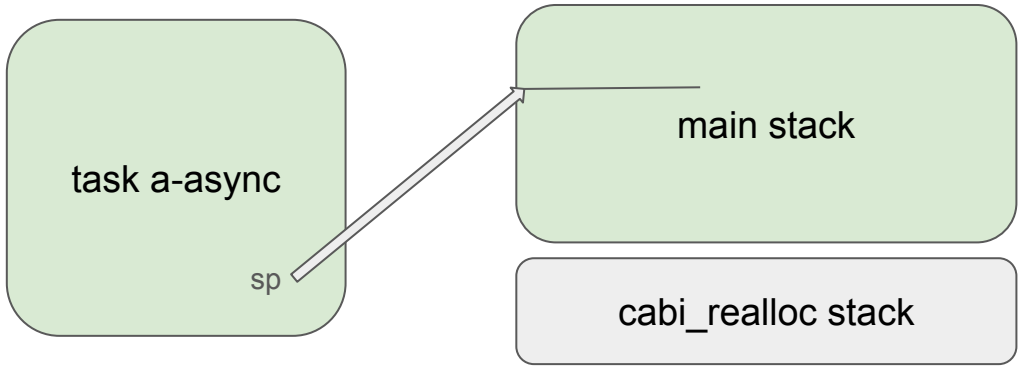
Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



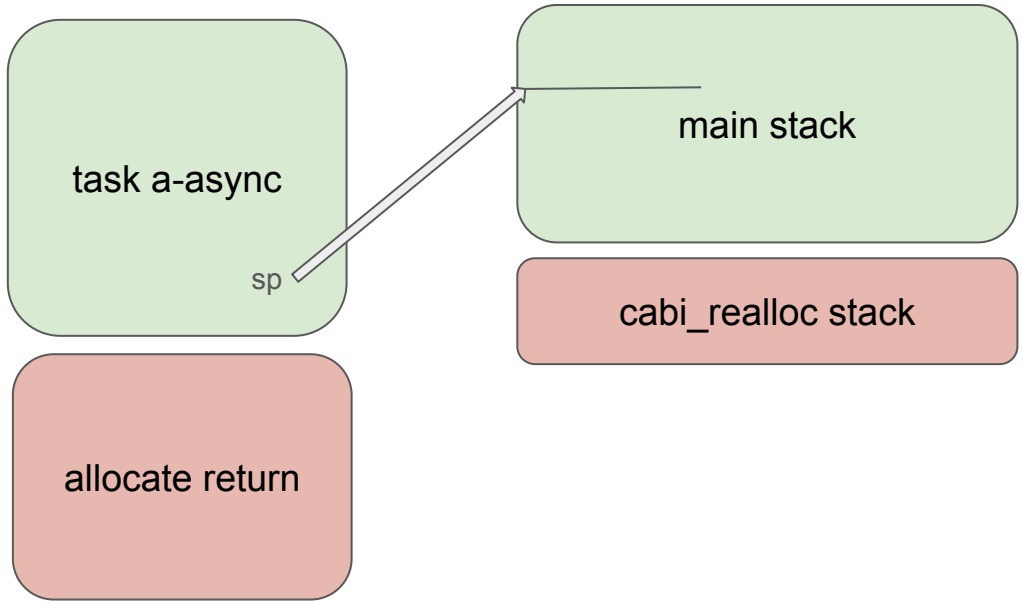
Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



Problem 1: stacks (part 2)

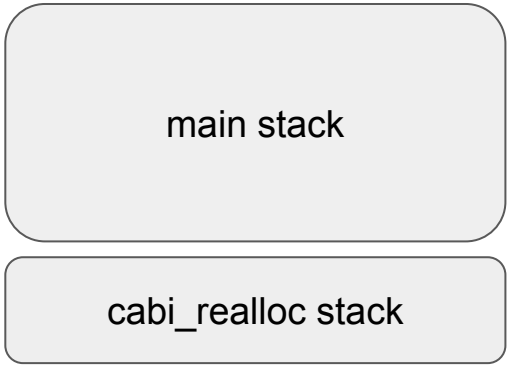
- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns



Problem 1: stacks (part 2)

- Start a-async
- Block in import
- Start b-sync
- Finish b-sync
- Allocate a-async's return
- a-async returns





Problem 2: thread-local storage (TLS)

- TLS historically is basically just a global
  - Why do something special when there's just one thread?
- Sy's work in LLVM makes the TLS base behave similarly to stacks

```
(module
  (import "env" "__wasm_get_tls_base" (func (result i32)))
  (import "env" "__wasm_set_tls_base" (func (param i32)))
)
```

- Prior to WASIp3 all invocations of exports share the same TLS

Problem 2: thread-local storage (TLS)

- Solution: add another context slot!
  - `context.{get,set} 1` - part of component-model-threading
- LLVM/`wasm-ld` expose TLS block size/alignment for one module
  - Accesses are a static offset from a module-local base pointer
- TLS configured during `__wasm_task_hook` like the stack pointer
  - All calls to exports share the same TLS block
- Newly-spawned threads get their own fresh TLS block
- Two major problems:
  - Testing wasip3 before it's released
  - Shared-everything dynamic libraries

Problem 2: thread-local storage (TLS)

	Static linking	**Dynamic libraries**
Without coop threads	`global.{get,set}`	
With coop threads	`context.{get,set} 1`	

Problem 2: thread-local storage (TLS)

- Libc needs to know all the dynamic libraries for spawning threads
- One build of libc needs to work for both static and dynamic linking
- Each dynamic library needs to get its own private base pointer
- Solution: add indirection!
  - Also leverage the `wasm-component-ld` linking stage
- "Real" TLS base is a list-of-per-library-TLS-base-pointers
  - Supports sparse per-library TLS for main thread
  - Spawned threads allocate one big block, base pointers first pointing later in block
- `wasm-component-ld` synthesizes `__wasm_{get,set}_tls_base` functions
  - No threads & static - synthetic functions to `global.{get,set}`
  - Threads & static - direct `context.{get,set} 1` intrinsics
  - No threads & dynamic - synthetic functions to `global.get` "real" base then load/store
  - Threads & dynamic - synthetic functions to `context.get 1` the "real" base then load/store

Problem 2: thread-local storage (TLS)

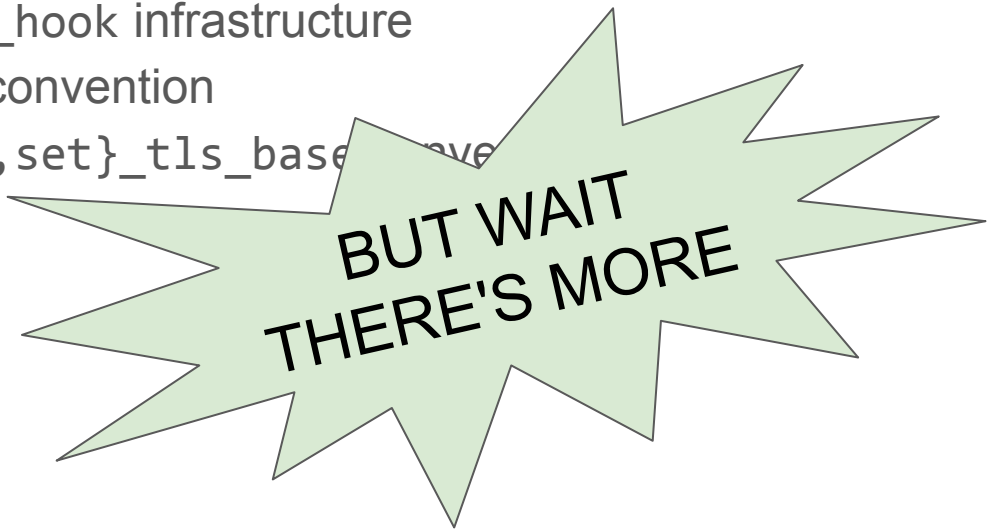
	Static linking	**Dynamic libraries**

Without coop threads	`global.{get,set}`	`global.get`
		`i32.{load,store}`
		`offset=N`

With coop threads	`context.{get,set} 1`	`context.get 1`
		`i32.{load,store}`
		`offset=N`

Let's add threads! 🧵

- Cooperative threading brings in new component model intrinsics
  - `thread.new-indirect`
  - `thread.{index,resume-later,suspend,yield}`
  - `thread.{suspend,yield}-then-{resume,promote}`
- Implement `__wasm_task_hook` infrastructure
- Implement stack pointer convention
- Implement `__wasm_{get,set}_tls_base`
- ...



Problem 3: futex

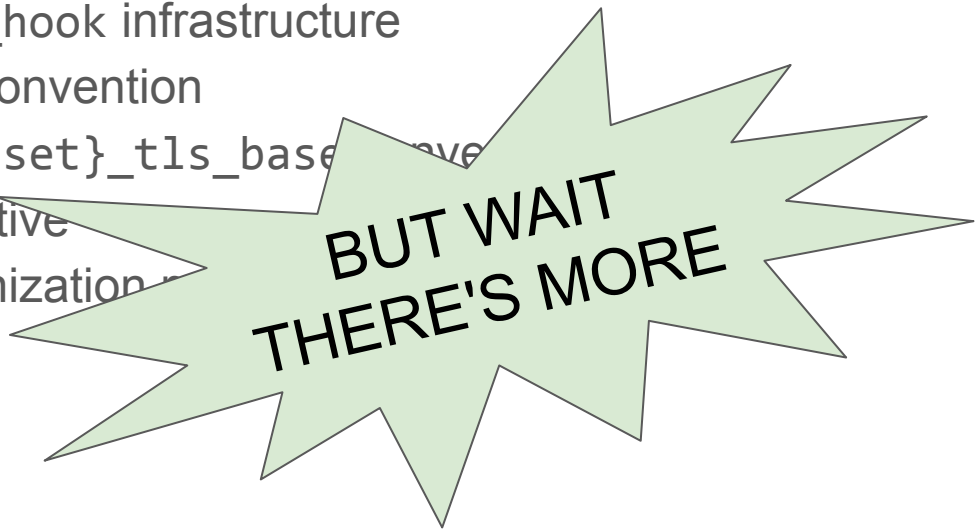
- POSIX has a fair number of synchronization primitives
  - Semaphores, barriers, N kinds of mutexes, rwlocks, condvars, ...
- Wasi-libc now has three different threading models
  - no threads
  - `wasm32-wasip1-threads`
  - coop threads
- Being POSIX-compliant for all of these is tricky
- ... but everything bottoms out in futex!

Problem 3: futex

- Sy also implemented a new primitive for synchronizing/blocking threads
  - Similar to `SYS_futex` on Linux, but functions in `wasi/libc.h`
- Wasi-libc has three implementations of futex
  - no threads - trap on blocking
  - wasi-threads - just use `memory.atomic.wait32`
  - coop threads - gob of code doing coop things
- POSIX primitives all inherited from upstream Musl
- Available for use in other libraries/languages
  - Rust doesn't use POSIX synchronization primitives (you might not want to either)

Let's add threads! 🧵

- Cooperative threading brings in new component model intrinsics
  - `thread.new-indirect`
  - `thread.{index,resume-later,suspend,yield}`
  - `thread.{suspend,yield}-then-{resume,promote}`
- Implement `__wasm_task_hook` infrastructure
- Implement stack pointer convention
- Implement `__wasm_{get,set}_tls_base`
- Implement futex-like primitive
- Fill out language synchronization
- ...

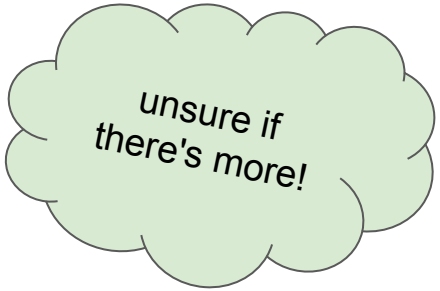


Problem 4: component-model task context

- Task context slot 0 is now the stack pointer on wasip3...
- ... but that was previously used by bindings generators as per-task data
  - Async exports akin to `Box<dyn Future>`, and that was stored here previously
- Add another slot?
  - Makes context switches that much more expensive
  - Not a scalable solution to every time this problem comes up
- Libc-based solution
  - `__wasilibc_task_context_{get,set}` - just one pointer
  - context slot 0 is BOTH the stack pointer and task-specific data
  - meaning changes if a task is running or suspended

Let's add threads! 🧵

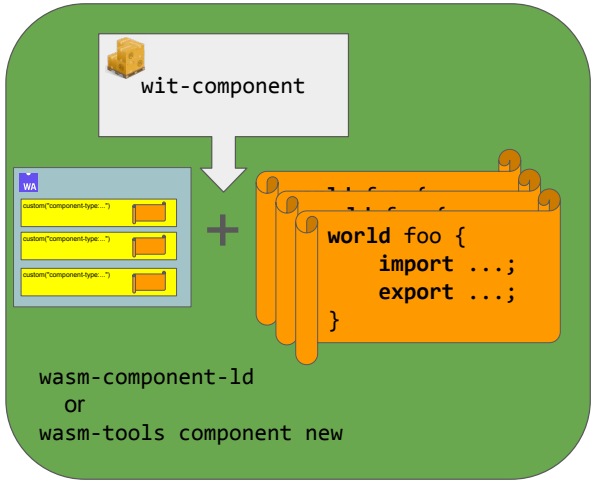
- Cooperative threading brings in new component model intrinsics
  - `thread.new-indirect`
  - `thread.{index,resume-later,suspend,yield}`
  - `thread.{suspend,yield}-then-{resume,promote}`
- Implement `__wasm_task_hook` infrastructure
- Implement stack pointer convention
- Implement `__wasm_{get,set}_tls_base` convention
- Implement futex-like primitive
- Fill out language synchronization primitives
- Fix task-local storage in bindings generators
- ...



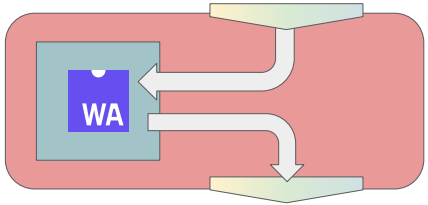
Status of coop threads

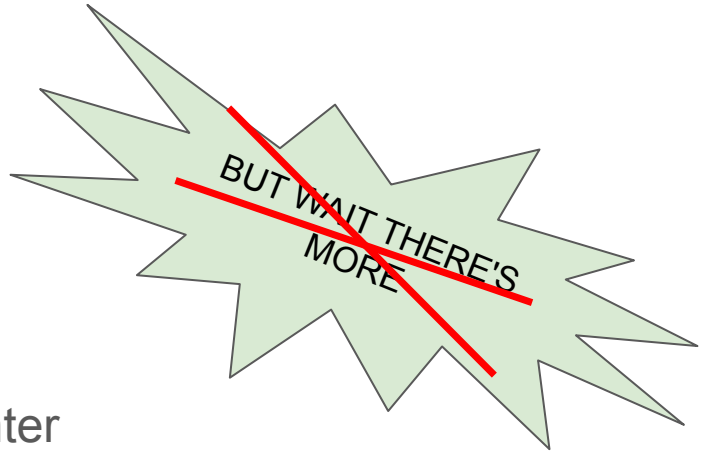
- Not yet stable/released, but you can test right this red hot second
  - Extra flags needed for C/C++
  - Extra flags needed for Rust
- Passes as many test suites as we can find
  - open-posix-test-suite
  - Rust's integration tests
  - Python's integration tests
  - componentize-py spawning threads
- No known issues at this time, but you may be able to enlighten us!

Under the hood of wasip3 components



=





- Problem: stack pointer
- Problem: TLS
- Problem: futex
- Problem: per-task context
- Fun fact: dynamic libraries
- Fun fact: task hooks